

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ
НАПРАВЛЕНИЕ ПРОГРАММНАЯ ИНЖЕНЕРИЯ
ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА СИСТЕМНОЕ И ПРИКЛАДНОЕ
ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

ОТЧЕТ ПО РЕШЕНИЮ ЗАДАЧ БЛОКА №3
курса «Алгоритмы и структуры данных»

Выполнил студент:

Исаева А.А.

группа: Р3209

Преподаватель:

Косяков М. С.,

Смирнов В. И.

Санкт-Петербург, 2026 г.

Содержание

1. Задача 9	3
1. Листинг кода	3
2. Описание решения	5
3. Обоснование	5
4. Сложность	5
2. Задача 10	5
1. Листинг кода	5
2. Описание решения	6
3. Обоснование	7
4. Сложность	7
3. Задача 11	7
1. Листинг кода	7
2. Описание решения	9
3. Обоснование	9
4. Сложность	10
4. Задача 12	10
1. Листинг кода	10
2. Описание решения	11
3. Обоснование	11
4. Сложность	11
5. Вывод	11

1. Задача 9

1. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <set>
4 #include <vector>
5
6 using namespace std;
7
8 int n, k, p;
9
10 vector<int> needed_cars;
11 vector<vector<int>> needed_cars_positions;
12 set<pair<int, int>> floor_cars;
13 vector<int> car_next_use;
14
15 int find_next_use(int car, int current) {
16     vector<int>& vec = needed_cars_positions[car];
17     auto it = upper_bound(vec.begin(), vec.end(),
18         current);
19     if (it == vec.end())
20         return p + 1;
21     return vec[it - vec.begin()];
22 }
23
24 bool is_on_floor(int car) {
25     return car_next_use[car] != -1;
26 }
27
28 void remove_from_floor(int car) {
29     floor_cars.erase({car_next_use[car], car});
30     car_next_use[car] = -1;
31 }
32
33 void add_to_floor(int car, int step) {
34     int next = find_next_use(car, step);
35     floor_cars.insert({next, car});
36     car_next_use[car] = next;
37 }
38
39 void evict_worst() {
```

```

39     int victim = prev(floor_cars.end())->second;
40     remove_from_floor(victim);
41 }
42
43 void refresh_on_floor(int car, int step) {
44     remove_from_floor(car);
45     add_to_floor(car, step);
46 }
47
48 int main() {
49     cin >> n >> k >> p;
50     needed_cars.resize(p);
51     needed_cars_positions.resize(n + 1);
52     car_next_use.assign(n + 1, -1);
53
54     for (int i = 0; i < p; i++) {
55         cin >> needed_cars[i];
56         needed_cars_positions[needed_cars[i]].push_back(i);
57     }
58
59     int count_operations = 0;
60     for (int i = 0; i < p; i++) {
61         int car = needed_cars[i];
62         if (is_on_floor(car)) {
63             refresh_on_floor(car, i);
64         } else {
65             count_operations++;
66             if ((int)floor_cars.size() == k) {
67                 evict_worst();
68             }
69             add_to_floor(car, i);
70         }
71     }
72     cout << count_operations;
73     return 0;
74 }

```

Листинг 1: Листинг кода задачи №9

1. 2. Описание решения

Симуляция работы кэша фиксированного размера k при последовательности запросов к данным. Используется оптимальная стратегия вытеснения: при необходимости удаляется элемент, следующее обращение к которому произойдёт позже всего (или не произойдёт вовсе). Для каждого запроса, если данные уже в кэше — обновляется информация о следующем использовании; если нет — фиксируется промах, при заполненном кэше удаляется «худший» элемент, после чего новые данные добавляются.

1. 3. Обоснование

- Алгоритм Беладди является оптимальным для задачи минимизации числа промахов кэша, если известна вся последовательность обращений.
- Вытеснение элемента с максимальным временем следующего использования гарантирует, что удаление не ухудшит будущие запросы сильнее, чем любое другое.
- Использование множества `floor_cars`, упорядоченного по времени следующего использования, позволяет за $O(\log k)$ выбирать жертву и обновлять состояние.

1. 4. Сложность

Время: $O(p \log k)$ — для каждого из p запросов выполняются операции вставки/удаления в сбалансированном дереве поиска, а также бинарный поиск следующего использования за $O(\log p)$ (предвычислен заранее).

Память: $O(n + p)$ — хранение списков позиций для каждой машины и вспомогательных структур.

2. Задача 10

2. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <set>
4 #include <vector>
5
6 using namespace std;
7
8 int main() {
```

```

9   int n;
10  string symbol;
11  vector<int> goblin_queue;
12  int goblin;
13  int goblin_index;
14  cin >> n;
15  for (int i = 0; i < n; i++) {
16      cin >> symbol;
17      if (symbol == "-") {
18          cout << goblin_queue[0] << "\n";
19          goblin_queue.erase(goblin_queue.begin());
20      }
21      if (symbol == "*" || symbol == "+") {
22          cin >> goblin;
23          if (symbol == "*") {
24              if (goblin_queue.size() % 2 == 0) {
25                  goblin_index = goblin_queue.size() / 2;
26                  goblin_queue.insert(goblin_queue.begin() +
goblin_index, goblin);
27              } else {
28                  goblin_index = goblin_queue.size() / 2 + 1;
29                  goblin_queue.insert(goblin_queue.begin() +
goblin_index, goblin);
30              }
31          } else {
32              goblin_queue.push_back(goblin);
33          }
34      }
35  }
36  return 0;
37 }

```

Листинг 2: Листинг кода задачи №10

2. 2. Описание решения

Моделируется очередь гоблинов с двумя операциями добавления: обычное добавление в конец (символ +) и добавление в середину (символ *), где новая позиция зависит от чётности текущей длины очереди. Для извлечения из начала используется символ - с выводом удаляемого элемента. Все операции выполняются над вектором, вставка в середину реализована через insert.

2. 3. Обоснование

- Прямая симуляция с использованием динамического массива позволяет корректно воспроизвести поведение очереди с дополнительной операцией вставки в позицию, зависящую от размера.
- Правило выбора индекса при * соответствует условию задачи: при чётной длине — после первой половины, иначе — после середины+1.

2. 4. Сложность

Время: $O(n^2)$ в худшем случае — каждая вставка в середину вектора требует сдвига элементов за $O(n)$, где n — количество операций.

Память: $O(n)$ — хранение очереди в векторе.

3. Задача 11

3. 1. Листинг кода

```
1 #include <iostream>
2 #include <map>
3 #include <set>
4 #include <vector>
5 using namespace std;
6
7 struct AllocResult {
8     long long start;
9     long long size;
10    bool success;
11 };
12
13 map<long long, long long> by_start;
14 map<long long, long long> by_end;
15 set<pair<long long, long long>> free_set;
16 map<int, AllocResult> history;
17
18 void add_free_block(long long start, long long size)
19 {
20     long long end = start + size - 1;
21
22     auto left = by_end.find(start - 1);
23     if (left != by_end.end()) {
24         long long l_start = left->second;
```

```

24     long long l_size = by_start[l_start];
25     free_set.erase({l_size, l_start});
26     by_start.erase(l_start);
27     by_end.erase(l_start + l_size - 1);
28     start = l_start;
29     size += l_size;
30     end = start + size - 1;
31 }
32
33 auto right = by_start.find(end + 1);
34 if (right != by_start.end()) {
35     long long r_start = right->first;
36     long long r_size = right->second;
37     free_set.erase({r_size, r_start});
38     by_start.erase(r_start);
39     by_end.erase(r_start + r_size - 1);
40     size += r_size;
41 }
42
43 free_set.insert({size, start});
44 by_start[start] = size;
45 by_end[start + size - 1] = start;
46 }
47
48 long long alloc(long long k) {
49     auto it = free_set.lower_bound({k, 0});
50     if (it == free_set.end())
51         return -1;
52     long long size = it->first;
53     long long start = it->second;
54     free_set.erase(it);
55     by_start.erase(start);
56     by_end.erase(start + size - 1);
57     if (size > k) {
58         add_free_block(start + k, size - k);
59     }
60     return start;
61 }
62
63 void free_by_request(int t) {
64     if (!history[t].success)
65         return;
66     add_free_block(history[t].start, history[t].size);

```



```

67 }
68
69 int main() {
70     long long n;
71     int m;
72     cin >> n >> m;
73     add_free_block(1, n);
74     for (int i = 1; i <= m; ++i) {
75         long long x;
76         cin >> x;
77         if (x > 0) {
78             long long res = alloc(x);
79             cout << res << '\n';
80             history[i] = {res, x, res != -1};
81         } else {
82             free_by_request((int)(-x));
83         }
84     }
85     return 0;
86 }

```

Листинг 3: Листинг кода задачи №11

3. 2. Описание решения

Реализован аллокатор памяти, поддерживающий выделение блока заданного размера и освобождение блока по номеру запроса. Свободные блоки хранятся в трёх структурах: **free_set** (размер, начало) для поиска подходящего блока, **by_start** и **by_end** для быстрого объединения соседних свободных областей. При выделении ищется блок минимального подходящего размера, при освобождении — соседние блоки автоматически сливаются.

3. 3. Обоснование

- Использование **std::set** с нижней границей по размеру позволяет найти подходящий свободный блок за $O(\log m)$, где m — количество блоков.
- Объединение соседних блоков при освобождении гарантирует, что свободная память не фрагментируется, и большие запросы могут быть удовлетворены.
- Исторические данные о выделениях позволяют освобождать блок по номеру запроса, даже если он был выделен не полностью (при выделении части блока остаток возвращается в свободные).

3. 4. Сложность

Время: $O(m \log m)$ на все операции — каждая операция выделения или освобождения требует нескольких операций вставки/удаления из `std::set` и `std::map`.

Память: $O(m)$ — хранение свободных блоков и истории выделений.

4. Задача 12

4. 1. Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <set>
4 #include <vector>
5
6 using namespace std;
7
8 int main() {
9     int n, k;
10    cin >> n >> k;
11    vector<int> numbers(n);
12    for (int i = 0; i < n; i++) {
13        cin >> numbers[i];
14    }
15    multiset<int> k_numbers;
16    for (int i = 0; i < n; i++) {
17        if (i < k - 1) {
18            k_numbers.insert(numbers[i]);
19        } else {
20            if (i != k - 1) {
21                k_numbers.erase(k_numbers.find(numbers[i - k
22            ]));
23            }
24            k_numbers.insert(numbers[i]);
25            cout << *k_numbers.begin() << " ";
26        }
27    }
28    return 0;
29 }
```

Листинг 4: Листинг кода задачи №12

4. 2. Описание решения

Для каждого окна размера k в массиве из n целых чисел требуется вывести минимальный элемент. Используется структура `multiset`, в которую поочерёдно добавляются элементы. Как только размер окна достигает k , выводится текущий минимум (первый элемент мультисета). При сдвиге окна из мультисета удаляется элемент, выходящий за пределы окна.

4. 3. Обоснование

- `multiset` поддерживает упорядоченность элементов, поэтому минимум всегда доступен за $O(1)$ (итератор на начало).
- Удаление элемента по значению (с использованием итератора) и вставка выполняются за $O(\log k)$, что даёт общую сложность $O(n \log k)$.
- Алгоритм обрабатывает каждое окно ровно один раз, не требует дополнительной памяти на все окна.

4. 4. Сложность

Время: $O(n \log k)$ — на каждый из n элементов выполняется вставка и, возможно, удаление из мультисета.

Память: $O(k)$ — хранение элементов текущего окна в `multiset`.

5. Вывод

В ходе решения задач были применены различные алгоритмические парадигмы и структуры данных. Задача 9 продемонстрировала оптимальное управление кэшем (алгоритм Беладии) с использованием предвычисления следующих обращений и сбалансированного дерева для выбора жертвы. Задача 10 реализует симуляцию очереди с нестандартной вставкой «в середину», наглядно показывая работу с динамическими массивами. Задача 11 представляет собой полноценный аллокатор памяти с поддержкой слияния свободных блоков, использующий `set` и `map` для эффективных операций. Задача 12 решает задачу скользящего минимума с помощью мультисета, что является удобной альтернативой классическому алгоритму с деком. Таким образом, все четыре задачи иллюстрируют важность выбора подходящих структур данных для достижения оптимальной сложности.